

State e Memento

Padrões de Projeto - Aula #11

Prof. Dr. Carlos Melo

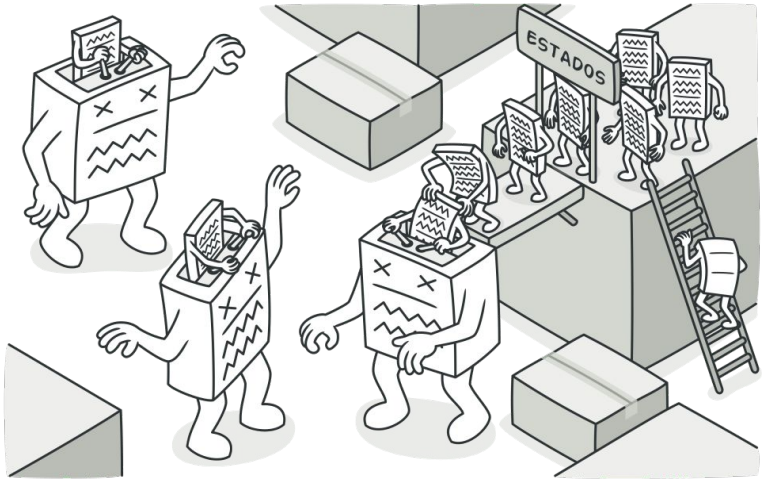
carlos.melo@upe.br

UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Padrões Comportamentais (parte #4)

State

Organiza comportamentos que variam conforme o estado atual de um objeto



Memento

Permite guardar e restaurar estados anteriores de um objeto, preservando seu encapsulamento



- State trata do comportamento no estado atual
- Memento trata da captura e restauração de estados anteriores

carlos.melo@upe.br

Objetivos de aprendizagem

- Identificar comportamentos e transições que dependem do estado de um objeto
- Implementar *State* e distingui-lo de *Strategy* pela intenção
- Implementar a captura e a restauração do estado de um editor
- Reconhecer quando cada padrão justifica a estrutura adicional

State - O Problema

- Um sistema de publicação de documentos
- Um documento pode estar em três estados:
  - Rascunho
  - Moderação
  - Publicado
- A mesma operação, `publicar()`, tem efeitos diferentes:
  - Em rascunho ela encaminha o documento para moderação
  - Em moderação o documento é publicado quando o usuário é administrador
  - Em publicado o documento é mantido publicado
- Na moderação, uma tentativa de publicação por outro perfil mantém o estado atual

State - Implementação com condicionais

```python
class Documento:
    def __init__(self) -> None:
        self.estado = "rascunho"

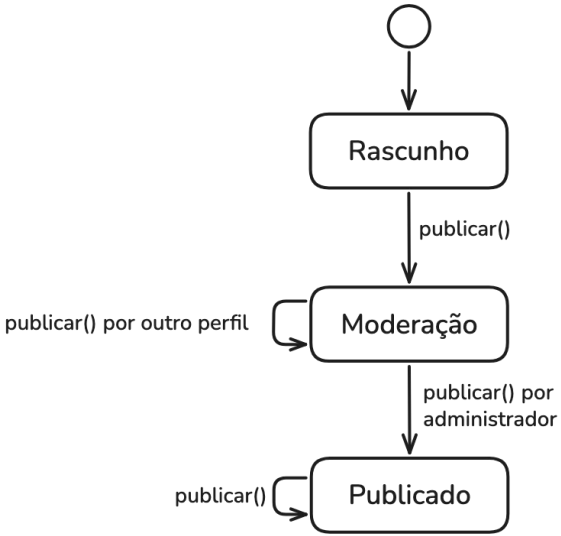
    def publicar(self, perfil: str) -> None:
        if self.estado == "rascunho":
            self.estado = "moderacao"

        elif self.estado == "moderacao":
            if perfil == "administrador":
                self.estado = "publicado"

        elif self.estado == "publicado":
            pass
```

- Essa implementação pode ser suficiente para um caso pequeno
- O problema aparece quando:
  - Várias operações repetem verificações do estado atual.
  - Novos estados exigem alterações em diferentes métodos.
  - As regras de comportamento e de transição ficam espalhadas pela classe

State - estados e transições do documento



- O documento mantém um estado ativo por vez
- A operação `publicar()` depende do estado atual e do perfil recebido
- As setas que retornam ao mesmo estado representam tentativas que mantêm esse estado

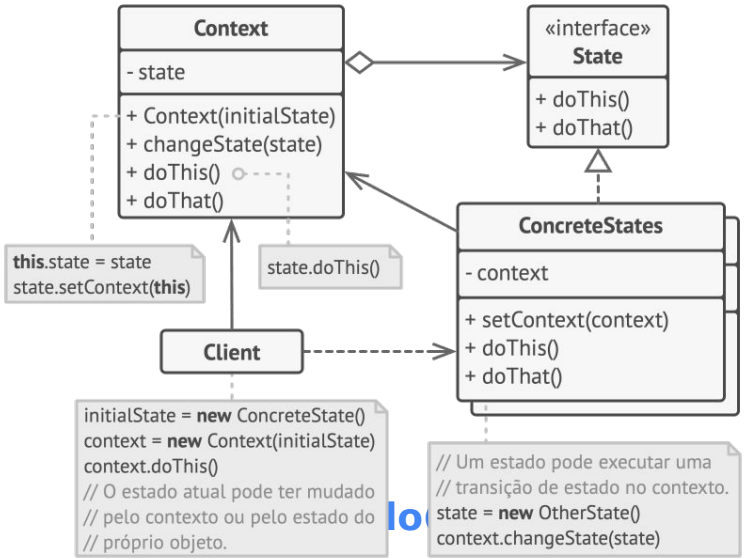
State - ideia e intenção

"Permitir que um objeto altere seu comportamento conforme seu estado interno muda"

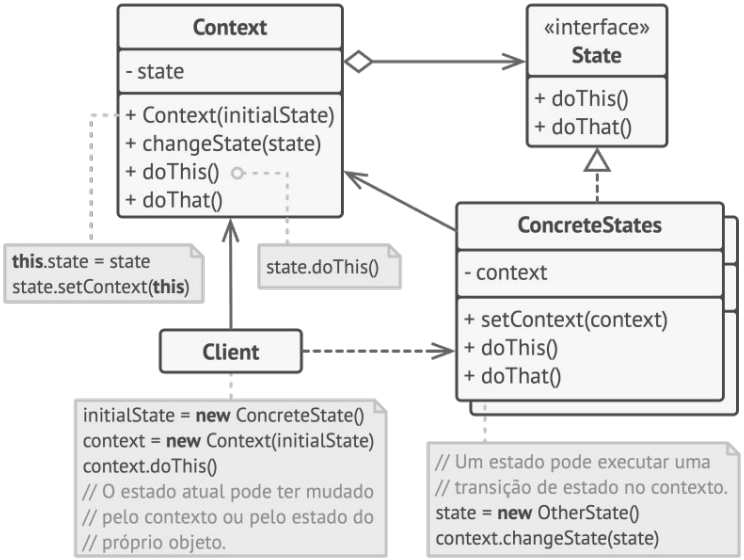
- No documento:
  - Cada estado terá uma classe própria
  - Todas as classes de estado oferecerão a operação `publicar()`
  - O `Documento` manterá uma referência ao estado ativo e delegará a operação
  - Uma transição substituirá o objeto de estado associado ao documento
- As regras de publicação ficam nas classes Rascunho, Moderacao e Publicado

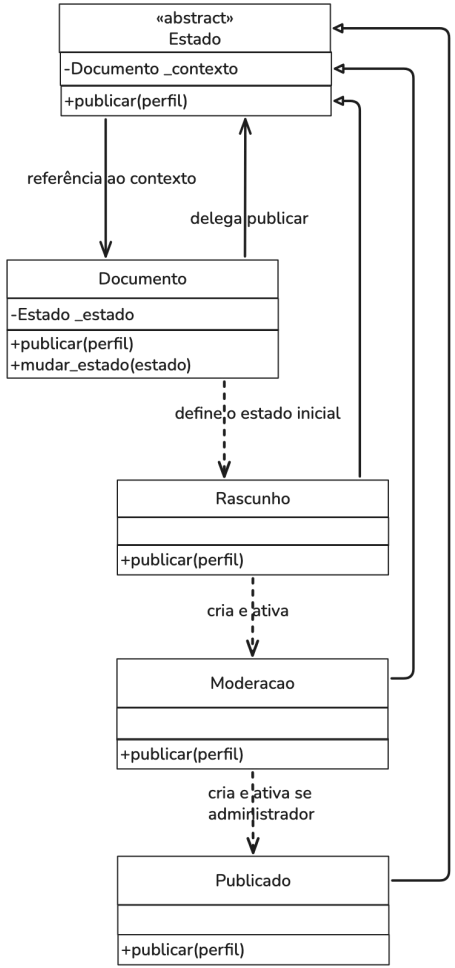
State - Estrutura

1. O **Context** mantém o estado ativo, delega operações e oferece uma operação para substituí-lo
2. **State** declara o contrato dos comportamentos que variam
3. **ConcreteStates** implementam esses comportamentos e podem manter uma referência ao contexto
4. Transições determinam o contexto e os **ConcreteStates** podem determinar a troca do estado ativo.



Aplicando ao nosso contexto





Aplicando ao nosso contexto - State

```python
from abc import ABC, abstractmethod


class Estado(ABC):
    def __init__(self, contexto: "Documento") -> None:
        self._contexto = contexto

    @abstractmethod
    def publicar(self, perfil: str) -> None:
        pass
```

- `_estado` guarda o objeto responsável pelo comportamento atual
- A propriedade estado permite consultar o nome desse estado
- `mudar_estado()` substitui o estado ativo
- `publicar()` delega a decisão ao estado atual

Aplicando ao nosso contexto - Context

```python
class Documento:
    def __init__(self) -> None:
        self._estado: Estado = Rascunho(self)

    @property
    def estado(self) -> str:
        return type(self._estado).__name__

    def mudar_estado(self, estado: Estado) -> None:
        self._estado = estado

    def publicar(self, perfil: str) -> None:
        self._estado.publicar(perfil)
```

- `_estado` guarda o objeto responsável pelo comportamento atual
- A propriedade estado permite consultar o nome desse estado
- `mudar_estado()` substitui o estado ativo
- `publicar()` delega a decisão ao estado atual

Aplicando ao nosso contexto - Concrete State

```python
class Rascunho(Estado):
    def publicar(self, perfil: str) -> None:
        self._contexto.mudar_estado(Moderacao(self._contexto))


class Moderacao(Estado):
    def publicar(self, perfil: str) -> None:
        if perfil == "administrador":
            self._contexto.mudar_estado(Publicado(self._contexto))


class Publicado(Estado):
    def publicar(self, perfil: str) -> None:
        pass
```

- `Rascunho`: encaminha o documento para moderação
- `Moderação`: permite a publicação quando o perfil é administrador
- `Publicado`: mantém o documento publicado
- Ao criar o próximo estado, passamos a referência para o mesmo documento

Aplicando ao nosso contexto - Execução

```
documento = Documento()
print(documento.estado)

documento.publicar("autor")
print(documento.estado)

documento.publicar("autor")
print(documento.estado)

documento.publicar("administrador")
print(documento.estado)

documento.publicar("autor")
print(documento.estado)
```

- O documento recebe chamadas ao mesmo método, mas o estado ativo determina o comportamento

Prós e contras do State

Prós

- Reúne o comportamento de cada estado em uma classe
- Reduz condicionais de estado espalhadas pelo Contexto
- Permite localizar as regras de cada estado com facilidade

Contras

- Aumenta a quantidade de classes
- Pode adicionar complexidade desnecessária a casos simples
- Estados podem depender de outros estados para realizar transições

Use:

- Quando o comportamento de um objeto depende do seu estado atual
- Quando as regras de comportamento e transição mudam com frequência
- Quando vários métodos repetem condicionais para verificar os mesmos estados

Memento - O Problema

- No editor de texto acompanharemos o texto e a posição do cursor

Momento	Texto	Cursor
Antes da edição	"Olá"	3
Depois de inserir " mundo"	"Olá mundo"	9
Depois de desfazer	"Olá"	3

- O cursor indica a posição de inserção:
  - `0` representa o início do texto
- Para desfazer, precisamos recuperar os valores anteriores
- Se o histórico copiar diretamente os atributos do editor, ficará dependente da sua estrutura interna

Memento - ideia e intenção

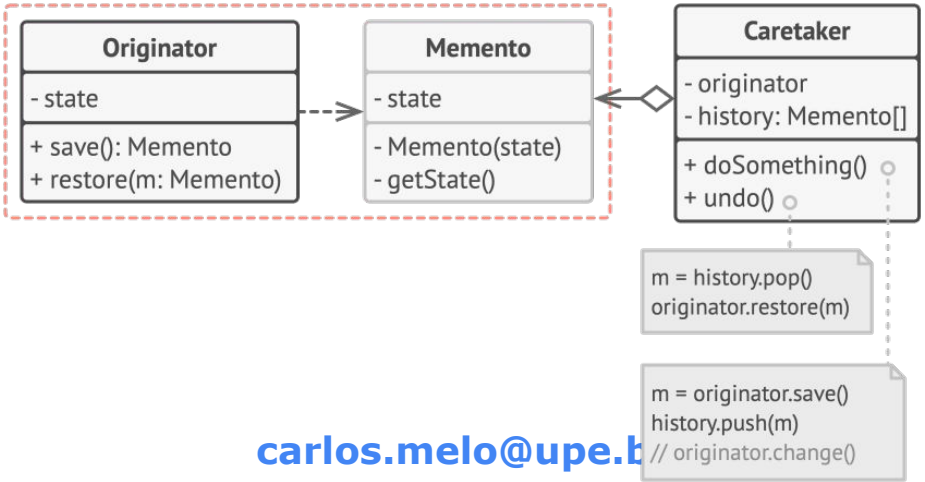
"Permitir salvar e restaurar o estado anterior de um objeto sem revelar os detalhes de sua implementação"

- Ideia central:
  - O próprio objeto cria um retrato do seu estado, chamado Memento
  - Outro objeto guarda esse retrato e pode solicitar sua restauração, sem precisar interpretar os dados armazenados
- As regras de publicação ficam nas classes Rascunho, Moderacao e Publicado

Memento - Estrutura baseada em classes aninhadas

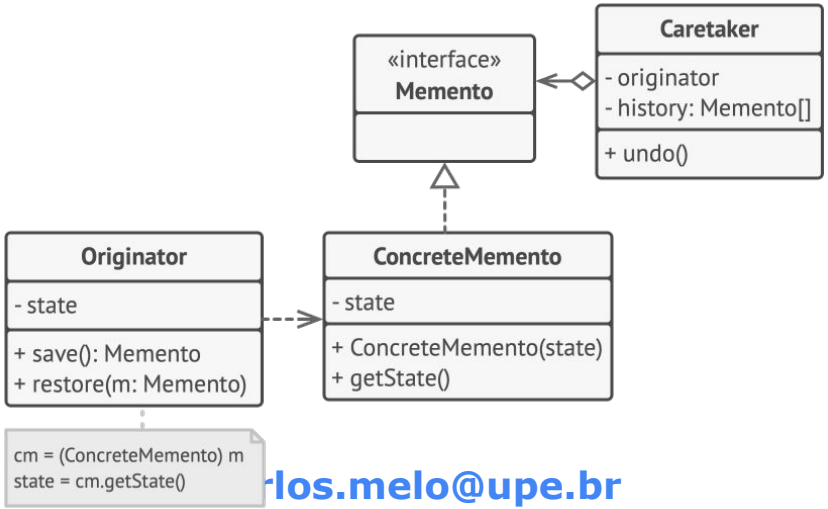
- A **Originadora** produz um retrato do próprio estado e sabe restaurá-lo
- A **Cuidadora** organiza o histórico desses retratos

1. Antes da edição, o histórico solicita um retrato ao editor
2. O editor cria um Memento com os valores atuais
3. A edição modifica o editor e o retrato permanece estável
4. Ao desfazer, o histórico devolve o retrato ao editor, que restaura seus valores

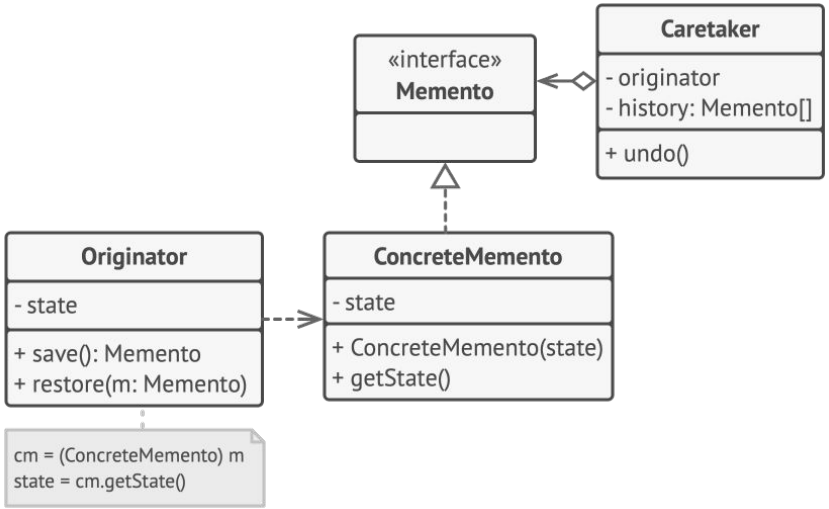


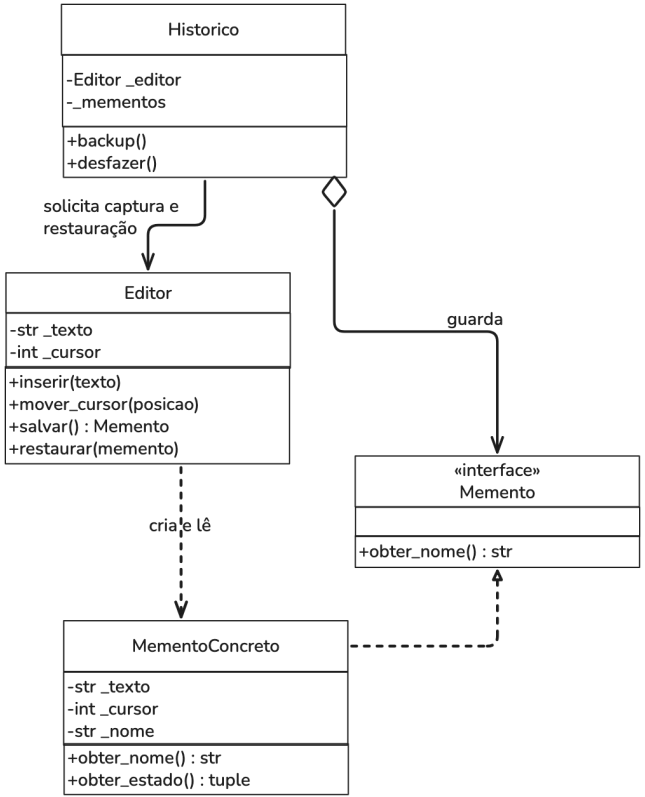
Memento - Estrutura baseada em uma interface intermediária

1. Na ausência de classes aninhadas, você pode restringir o acesso aos campos do memento ao estabelecer uma convenção para que cuidadoras possam trabalhar com um memento através apenas de uma interface intermediária explicitamente declarada
2. As originadoras podem trabalhar com um objeto memento diretamente, acessando campos e métodos declarados na classe memento



Aplicando ao nosso contexto





Aplicando ao nosso contexto - Memento

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass


class Memento(ABC):
    @abstractmethod
    def obter_nome(self) -> str:
        pass


@dataclass(frozen=True)
class MementoConcreto(Memento):
    _texto: str
    _cursor: int
    _nome: str = "Antes da edição"

    def obter_nome(self) -> str:
        return self._nome

    def obter_estado(self) -> tuple[str, int]:
        return self._texto, self._cursor
```

- `frozen=True` impede a reatribuição dos campos após a criação
- `obter_estado()` será utilizado pela Originadora, não pela Cuidadora

Aplicando ao nosso contexto - Originadora (#1)

```python
class Editor:
    def __init__(self, texto: str = "") -> None:
        self._texto = texto
        self._cursor = len(texto)

    @property
    def texto(self) -> str:
        return self._texto

    @property
    def cursor(self) -> int:
        return self._cursor

    def mover_cursor(self, posicao: int) -> None:
        if not 0 <= posicao <= len(self._texto):
            raise ValueError("Posição inválida.")
        self._cursor = posicao

    def inserir(self, texto: str) -> None:
        p = self._cursor
        self._texto = self._texto[:p] + texto + self._texto[p:]
        self._cursor += len(texto)
```

- O Editor mantém o texto e o cursor
- Inserir um texto também atualiza a posição do cursor

Aplicando ao nosso contexto - Originadora (#2)

```python
    def salvar(self) -> Memento:
        return MementoConcreto(self._texto, self._cursor)

    def restaurar(self, memento: Memento) -> None:
        if not isinstance(memento, MementoConcreto):
            raise TypeError("Memento incompatível.")
        self._texto, self._cursor = memento.obter_estado()
```

- `salvar()` captura os valores atuais em um novo retrato
- `restaurar()` recupera os valores desse retrato
- O próprio Editor conhece os dados necessários para realizar essas operações

Aplicando ao nosso contexto - Cuidadora

```python
class Historico:
    def __init__(self, editor: Editor) -> None:
        self._editor = editor
        self._mementos: list[Memento] = []

    def backup(self) -> None:
        self._mementos.append(self._editor.salvar())

    def desfazer(self) -> None:
        if self._mementos:
            self._editor.restaurar(self._mementos.pop())
```

- `backup()` solicita e guarda um retrato.
- `desfazer()` remove o último retrato e o devolve ao editor.
- Se a pilha estiver vazia, o editor permanece como está.

Aplicando ao nosso contexto - Execução

```
editor = Editor("Olá")
historico = Historico(editor)
print(editor.texto, editor.cursor)

historico.backup()
editor.inserir(" mundo")
print(editor.texto, editor.cursor)

historico.backup()
editor.inserir("!")
print(editor.texto, editor.cursor)

historico.desfazer()
print(editor.texto, editor.cursor)

historico.desfazer()
print(editor.texto, editor.cursor)
```

Prós e Contras do Memento

Prós

- O próprio objeto controla a captura e a restauração do seu estado
- A Cuidadora administra o histórico sem interpretar o estado armazenado

Contras

- Muitos retratos podem consumir bastante memória
- É necessário definir quando descartar retratos obsoletos

Use

- Quando for necessário restaurar um estado anterior de um objeto
- Quando copiar atributos por fora do objeto expuser detalhes da sua implementação
- Quando o histórico precisar ser administrado separadamente das regras do objeto



Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS